jaato TUI — User Manual

Generated by the jaato-tui-driven-tests harness

2026-06-26

Contents

Introduction

This manual documents the jaato TUI client (the `jaato` command) — the terminal interface for interacting with the jaato daemon. Sections are organized into:

- **Reference catalogs** (auto-generated): keybindings, slash commands, tools, profile knobs. These reflect the live framework state on the day this manual was built.
- **Feature walkthroughs** (auto-generated capture + operator-edited caption): one section per feature, showing what you see and what to do with it.
- **Appendix**: tear-down, troubleshooting, where to read the source.

This document is built by a self-documenting harness — the harness drives a real TUI session via tmux and captures each feature's screen state directly. The captures are real terminal output, not synthesized mockups.

If something in here looks wrong or stale, regenerate the manual:

```
cd jaato-tui-driven-tests
python -m harness all
```

The walk phase auto-starts a daemon on `/tmp/jaato.sock` if one isn't already running — same default behaviour as launching `jaato` directly.

Getting started — running the TUI

The TUI is launched with the `jaato` command. The daemon (the "server") is launched with `jaato-server`. Both come from the `jaato-tui` and `jaato-server` packages respectively; once the project's virtualenv is activated, both commands are on `PATH` — there is no need to invoke `python -m server` or `python rich_client.py` manually.

First run — auto-start

The simplest way to start a session is:

```
$ jaato
```

If no daemon is listening on the default IPC socket (`/tmp/jaato.sock`), the TUI **spawns one in the background** and connects to it. The default config directory is `.jaato/` in the current working directory; if that directory does not exist, the TUI will exit and ask you to scaffold it first:

```
$ jaato --init
```

`--init` writes a starter `.jaato/` next to the current directory with example agents, profiles, references, and an `.env` template. Edit those files, then re-run `jaato`.

Reconnecting to an existing session

Re-running `jaato` on a workspace where the daemon is already listening on `/tmp/jaato.sock` will **resume the default session** — the same conversation history, todos, plan, and budget you had in the previous TUI window. This is the expected behaviour when you close the TUI (Ctrl+D) and come back later: the daemon stays alive in the background and your context is waiting for you.

If you want to start fresh without losing the prior session, pass `--new-session`:

```
$ jaato --new-session
```

The default session is left intact on the daemon — you can switch back to it later from the TUI's session-management commands.

Connecting to a specific socket

The default IPC socket path is `/tmp/jaato.sock`. If you run the daemon on a different socket (for example to isolate workspaces or to run multiple daemons side-by-side), point the TUI at it explicitly:

```
$ jaato --connect /tmp/scratch.sock
```

When `--connect` is set, auto-start is disabled by default for any non-default path — the TUI assumes you already have the daemon running where you want it. Use `--no-auto-start` to opt out of auto-start unconditionally.

Running the daemon by hand

For most users, auto-start is sufficient. The cases where you want to run the daemon manually are:

- You want the daemon to outlive the TUI window (so reconnects are fast, even after a system reboot of the TUI's terminal multiplexer process).
- You want to expose the daemon over WebSocket so a remote client can connect.
- You want to inspect the daemon's logs in real time (`/tmp/jaato.log` rotates by default; the foreground daemon prints directly to its terminal).

Run the daemon manually with:

```
$ jaato-server --ipc-socket /tmp/jaato.sock --daemon
```

To also expose a WebSocket endpoint:

```
$ jaato-server --ipc-socket /tmp/jaato.sock --web-socket :8080 --daemon
```

The `--daemon` flag detaches the server from the current terminal. Without it the daemon runs in the foreground (useful for tailing logs).

Checking + stopping the daemon

```
$ jaato-server --status     # show pid, sockets, uptime
$ jaato-server --stop       # send graceful shutdown
```

Selecting a profile or agent at startup

Sessions can be created with a runtime **profile** (model + plugins + GC) or an **agent** (parameterized prompt + persona). Both are read from `.jaato/profiles/` and `.jaato/agents/` respectively. At startup, pass either or both:

```
$ jaato --profile researcher --agent code-explainer
```

`--new-session` is implied when `--profile` or `--agent` is set — profiles and agents only apply to fresh sessions. Existing sessions keep the profile they were created with.

A non-interactive one-shot

For scripting or piping output into another tool, ask the TUI for a single response and exit:

```
$ jaato --prompt "What changed in the latest commit?"
```

The TUI runs the prompt against the default session, prints the response, and exits. Auto-start still applies — this is a useful pattern for CI hooks and editor integrations that don't want a persistent terminal.

If you want to seed the session with an opening prompt but **stay interactive** afterwards, use `--initial-prompt` (`-i`) instead.

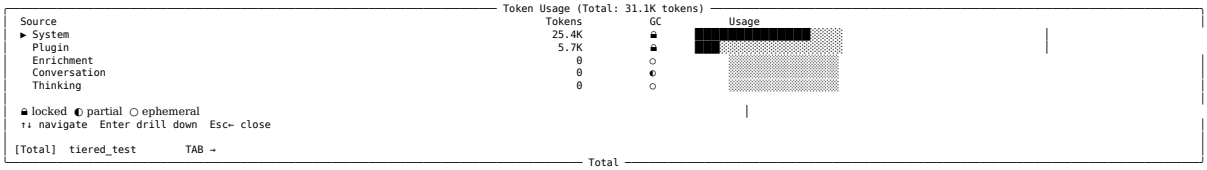
Ctrl+B — Budget panel (token usage by category)

View real-time token consumption across system, plugin, enrichment, conversation, and thinking categories.

The budget panel displays a live breakdown of token usage by source category. This is useful for understanding where your context budget is being consumed during a session. Each category shows the number of tokens used, a garbage-collection (GC) status indicator, and a visual bar representing usage as a proportion of the total.

The panel is organized as a hierarchical list. The top-level row shows the total token count for the entire session. Below that, each source category (System, Plugin, Enrichment, Conversation, Thinking) is listed with its own token count and status. You can navigate between categories using the arrow keys and drill down to see more details about locked or partial categories.

On screen:



Notes

- The **GC column** shows the garbage-collection status using three icons:
  - 🔒 **locked** — tokens cannot be freed; they are essential to the session
  - ◐ **partial** — tokens can be partially freed when space is needed
  - ○ **ephemeral** — tokens can be fully freed when space is needed
- The **Usage bar** is a visual representation of each category's token consumption relative to the total; filled blocks (█) represent used tokens, empty blocks (⣿) represent available capacity
- The ▶ **symbol** next to System indicates that category can be expanded to show sub-items; press Enter to drill down
- Total token count is displayed in the panel title and updated in real time as the session progresses
- **Keyboard shortcuts** (shown at the bottom of the panel):
  - ↑↓ — navigate between rows
  - **Enter** — drill down into a category for more detail
  - **Esc** — close the panel and return to the prompt
  - **Tab** — switch between different budget views (e.g., [Total] vs. session-specific)

Related sections

- initial-banner
- exit-menu

exit command — Detach / end-session menu

The exit command brings up a simple three-choice menu that lets you either detach the session, end it entirely, or return to the prompt.

When you type `exit` at the `User>` prompt, the TUI presents a small dialog with three options:

- **Detach** (`d`) – Keeps the session alive on the server but closes the local pane. You can re-attach later.
- **End** (`e`) – Terminates the session permanently, deleting all session data.
- **Return** (`r`) – Cancels the menu and brings you back to the normal prompt.

The menu is intentionally minimal so you can quickly choose the desired action without leaving the TUI.

On screen:

Notes

- The menu appears only when you type `exit` at the prompt.
- The `Choice [d/e/r]:` line waits for a single keystroke; pressing `Enter` after the letter submits the choice.
- Selecting `r` (Return) simply dismisses the menu and restores the `User>` prompt.

Related sections

- initial-banner
- permission-prompt-flow

Initial TUI state — banner + prompt

The TUI greets you with a header banner showing session info, an agent status line with model and budget, and helpful guidance text above the User> prompt.

When you start the Jaato TUI, the first screen shows a structured header containing key session information: your session ID, workspace path, toolblock status, and permission mode. Below that, the model provider and context budget are displayed, giving you immediate visibility into your current AI configuration and token availability. The main content area then presents helpful guidance text explaining the core capabilities—file editing, tab completion, available commands, and the connected AI model—before settling into the user prompt where you can begin entering commands.

On screen:

```
Session: 20260508_185659  |  Workspace: ~/Sources/Jaato-framework-and-examples/jaato-tui-driven-tests  |  Toolblocks: ▶ collapsed [Ctrl+T]  |  Permissions: ask
⁖ tiered_test
Provider: openrouter  |  Model: anthropic/claude-haiku-4.5  |  Context: 84% available (31K used, continuous →40%) [Ctrl+B for budget]

We now can edit files from the workspace panel
Tab completion enabled. Use @file for files, @@path for sandbox paths, %prompt for skills.
Type 'help' for commands, Ctrl+G for editor, Ctrl+F for search.
Connected to openrouter/anthropic/claude-haiku-4.5 (OpenRouter API key (JAATO_OPENROUTER_API_KEY))
Session created: Session 2026-05-08 18:56 (20260508_185659)

User>
```

Notes

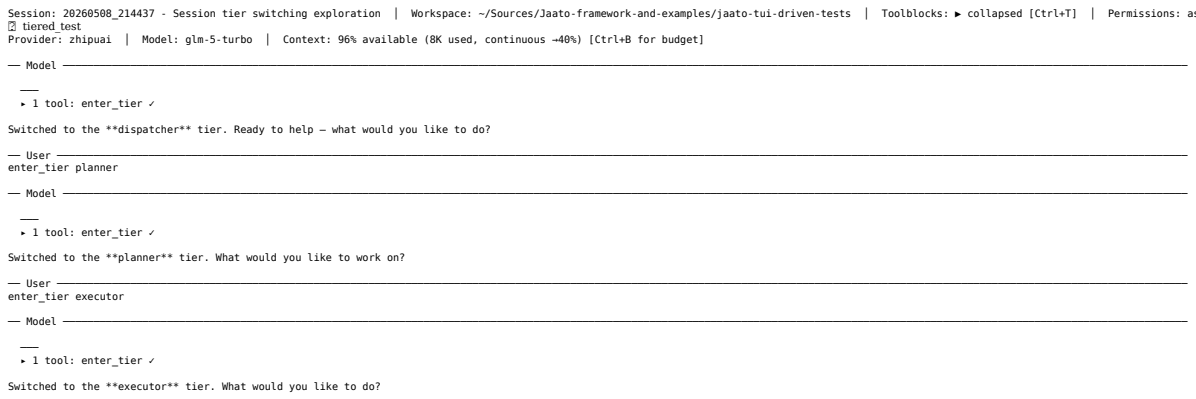
- The top line displays the session ID, workspace path, toolblock state, and permission mode at a glance
- The second line shows the agent status with a spinner (⁖) that animates while working, plus the current task name
- The third line reports the AI provider, model name, and context availability as a percentage; press Ctrl+B to see detailed budget breakdown
- The context budget shows both used tokens and a continuous budget indicator (e.g., `31K used, continuous →40%`)
- The guidance text includes tab completion syntax with `@file`, `@@path`, and `%prompt` prefixes for different completion types
- Additional lines confirm the connected model and API key source, plus the session creation timestamp
- The prompt line (`User>`) is where you enter your first command or question

Model Tier Switching

Switch between planner, dispatcher, and executor tiers to adapt the agent's reasoning style and tool access.

The jaato TUI lets you change the agent's "tier" — a mode that determines how the agent thinks and which tools it can use. Each tier has a different focus: planner for deep reasoning, dispatcher for coordination, and executor for mechanical tool execution. Use `enter_tier <name>` to switch between them, or type `list` to see available tiers and their purposes. The tier affects what tools are available and how the agent responds to your requests.

On screen:



Notes

- Three tiers are available: planner, dispatcher, and executor — each with a different focus and tool access
- Use `enter_tier <name>` to switch tiers (e.g., `enter_tier dispatcher`)
- Type `list` to see available tiers and their descriptions
- The tier determines which tools the agent can invoke and how it responds
- Switching to a tier you're already on shows a confirmation message
- The status indicator (□, ⁝., ⁘) changes based on the current tier

Related sections

- `ctrl-b-budget-panel` — Ctrl+B — Budget panel (token usage by category)
- `exit-menu` — Exit Menu
- `initial-banner` — Initial TUI state — banner + prompt
- `permission-prompt-flow` — Permission Prompt Flow

Permission Prompt Flow

When the model requests a tool execution, the TUI displays a permission prompt allowing you to approve, deny, or set a policy for the tool call.

When the model attempts to execute a tool (such as running a shell command), the TUI pauses and displays a permission prompt. This prompt shows the tool name, the arguments it will receive, and a set of response options. You can allow the tool to run once, deny it, or set a session-wide policy for how similar requests should be handled in the future.

The permission system gives you fine-grained control over what the model can do. Each response option has a different scope and memory behavior: some apply only to the current request, others persist for the rest of the session, and some can be whitelisted or blacklisted permanently. This design prevents accidental or unwanted tool executions while allowing you to streamline repeated operations.

On screen:

```
🔒 Permission required
   Tool: cli_based_tool
   Args: {'command': 'pwd'}
   [yes]  [no] [always] [turn] [idle] [once] [never] [all] [comment] [allow-comment] [edit]  ⇥ cycle  ↵
```

After selecting an option, the menu expands to show all available choices:

```
y/yes             Allow this tool execution
n/no              Deny this tool execution
a/always          Allow and whitelist for session
t/turn            Allow remaining tools this turn
i/idle            Allow until session goes idle
once/once         Allow once without remembering
never/never       Deny and blacklist for session
all/all           Allow all future requests in session
```

Notes

- The permission prompt appears at the bottom of the TUI pane whenever the model prepares to execute a tool.
- The prompt displays the tool name and its arguments so you can verify the intended action before approving.
- Each response option has a distinct scope: single request, current turn, session duration, or permanent session policy.
- Once you respond, the tool either executes (if approved) or is skipped (if denied), and the model continues with its response.
- The `[once]` indicator appears next to tool blocks to show which permission scope was applied to that execution.
- You can cycle through options using Tab (⇥) before pressing Enter to confirm your choice.

Related sections

- `initial-banner` — Initial TUI state and session setup
- `ctrl-b-budget-panel` — Token budget and usage tracking